Kohonen Neural Network (Self-Organizing Map)

Moin Mostakim

April 19, 2025

Introduction to Kohonen Neural Network

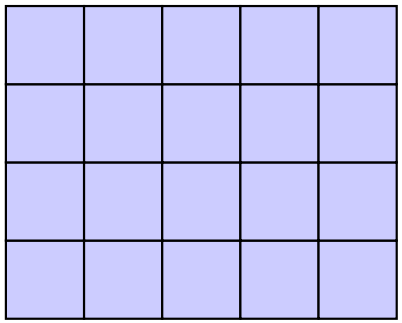
- Developed by Teuvo Kohonen in the 1980s.
- Unsupervised learning neural network.
- Projects high-dimensional data into a lower-dimensional (typically 2D) map.
- Preserves topological properties of the input space.
- Known as Self-Organizing Map (SOM).

SOM Architecture

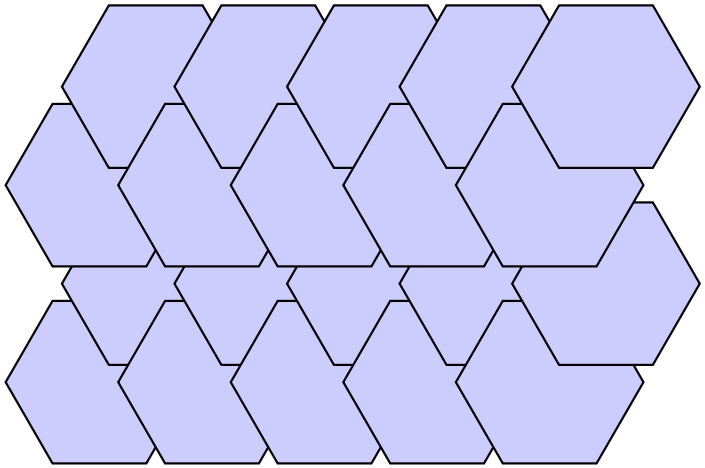
- Consists of input layer and a 2D grid of neurons (output layer).
- Each neuron is associated with a weight vector of the same dimension as input.
- Grid can be:
  - **Rectangular**
  - **Hexagonal**

SOM Structure - Rectangular vs. Hexagonal

Rectangular Grid

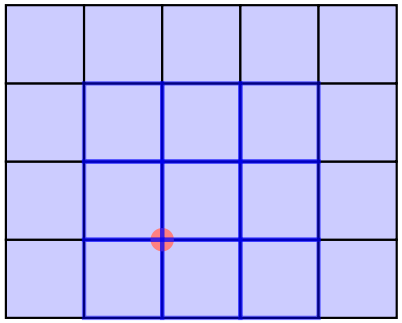


Hexagonal Grid

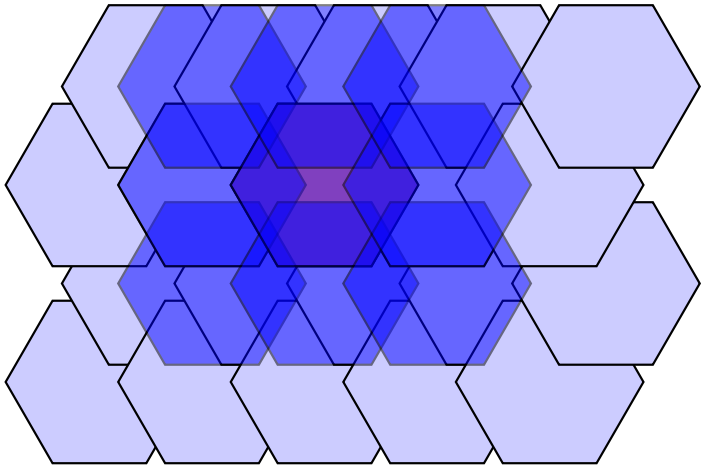


SOM Neighborhood Influence

Rectangular Grid Neighborhood Influence



Hexagonal Grid Neighborhood Influence



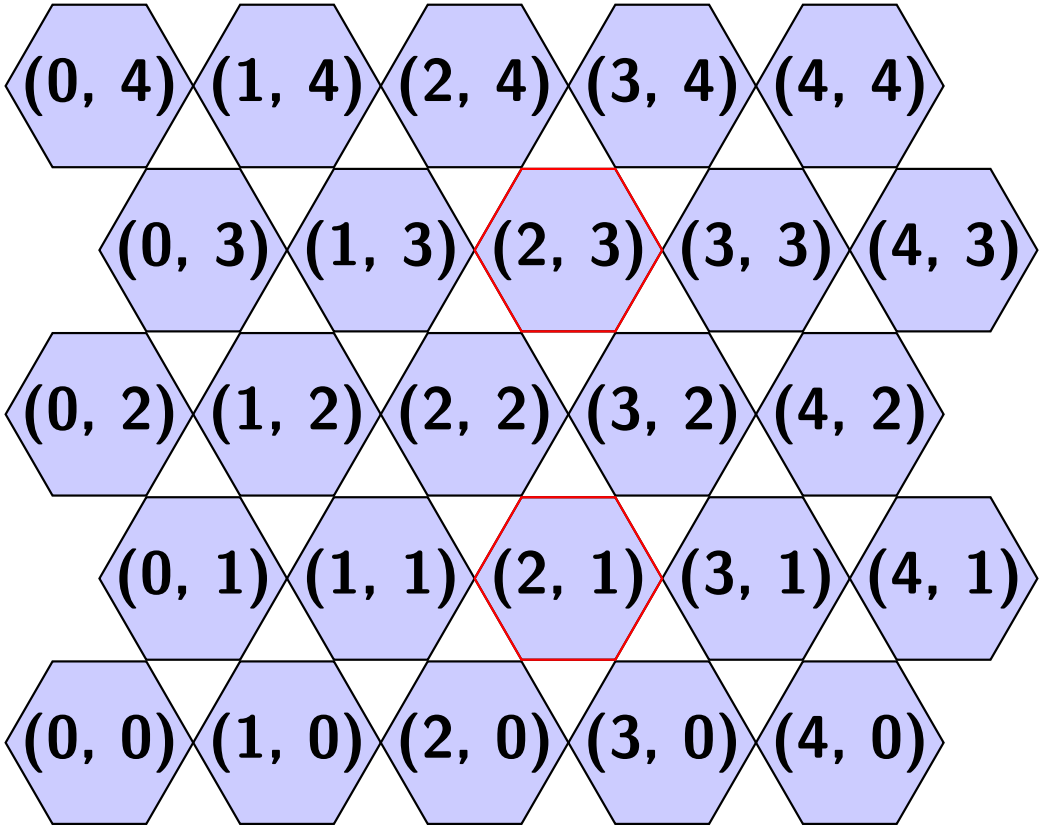
Neighbors in Rectangular Grid 5X5

Rectangular Grid with Coordinates and Neighbors

(0, 4)	(1, 4)	(2, 4)	(3, 4)	(4, 4)
(0, 3)	(1, 3)	(2, 3)	(3, 3)	(4, 3)
(0, 2)	(1, 2)	(2, 2)	(3, 2)	(4, 2)
(0, 1)	(1, 1) BMU	(2, 1)	(3, 1)	(4, 1)
(0, 0)	(1, 0)	(2, 0)	(3, 0)	(4, 0)

Neighbors in Hexagonal Grid 5X5

Hexagonal Grid with Coordinates and Neighbors



Derivation of Rectangular Distance Formula

We start with the standard **Euclidean distance** formula in a 2D grid.

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Where: - (x_i, y_i) and (x_j, y_j) are the coordinates of the two points on the grid. - i, j represent the grid coordinates of the two points. For the **rectangular grid**, we define (x, y) coordinates for each point as integers, and the BMU is located at $(2, 2)$. Hence, the distance between the BMU and any other point (i, j) is calculated as:

$$d_{ij} = \sqrt{(i - 2)^2 + (j - 2)^2}$$

This is the **Euclidean distance** between two points in the rectangular grid.

Example Calculation of Rectangular Distance

Let's now calculate the distance for a few grid points from the BMU at $(2, 2)$.
For the point $(0, 0)$:

$$d_{00} = \sqrt{(0-2)^2 + (0-2)^2} = \sqrt{4+4} = \sqrt{8} = 2.83$$

For the point $(1, 1)$:

$$d_{11} = \sqrt{(1-2)^2 + (1-2)^2} = \sqrt{1+1} = \sqrt{2} = 1.41$$

For the point $(3, 3)$:

$$d_{33} = \sqrt{(3-2)^2 + (3-2)^2} = \sqrt{1+1} = \sqrt{2} = 1.41$$

Thus, the Euclidean distance between any grid point and the BMU can be computed using this formula.

Rectangular Grid - 5x5 Grid Calculation

Best Matching Unit (BMU) at position $(2, 2)$

- Distance formula:

$$d_{ij} = \sqrt{(i-2)^2 + (j-2)^2}$$

- Calculation for all neurons:

$d_{00} = 2.83$	$d_{01} = 2.24$	$d_{02} = 2.00$	$d_{03} = 2.24$	$d_{04} = 2.83$
$d_{10} = 2.24$	$d_{11} = 1.41$	$d_{12} = 1.00$	$d_{13} = 1.41$	$d_{14} = 2.24$
$d_{20} = 2.00$	$d_{21} = 1.00$	$d_{22} = 0.00$	$d_{23} = 1.00$	$d_{24} = 2.00$
$d_{30} = 2.24$	$d_{31} = 1.41$	$d_{32} = 1.00$	$d_{33} = 1.41$	$d_{34} = 2.24$
$d_{40} = 2.83$	$d_{41} = 2.24$	$d_{42} = 2.00$	$d_{43} = 2.24$	$d_{44} = 2.83$

Derivation of Hexagonal Distance Formula

For a **hexagonal grid**, the distance calculation is slightly different. In a hexagonal grid, the distance between two points takes into account the structure of the hexagons, and we use the following formula:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 - (x_i - x_j)(y_i - y_j)}$$

Where: - (x_i, y_i) and (x_j, y_j) are the coordinates of the two points on the grid. - This formula accounts for the fact that the grid is not a simple square grid, but instead hexagonal, so there's a slight adjustment to the calculation.

Example Calculation of Hexagonal Distance

Let's now calculate the distance for a few grid points from the BMU at $(2, 2)$ in a hexagonal grid.

For the point $(0, 0)$:

$$d_{00} = \sqrt{(0-2)^2 + (0-2)^2 - (0-2)(0-2)} = \sqrt{4+4-4} = \sqrt{4} = 2.00$$

For the point $(1, 1)$:

$$d_{11} = \sqrt{(1-2)^2 + (1-2)^2 - (1-2)(1-2)} = \sqrt{1+1-1} = \sqrt{1} = 1.00$$

For the point $(3, 3)$:

$$d_{33} = \sqrt{(3-2)^2 + (3-2)^2 - (3-2)(3-2)} = \sqrt{1+1-1} = \sqrt{1} = 1.00$$

Thus, the hexagonal grid distance accounts for the geometric difference between hexagonal and rectangular grids.

Hexagonal Grid - 5x5 Grid Calculation

Best Matching Unit (BMU) at position $(2, 2)$

- Distance formula for hexagonal grid:

$$d_{ij} = \sqrt{(i-2)^2 + (j-2)^2 - (i-2)(j-2)}$$

- Calculation for all neurons:

$d_{00} = 2.65$	$d_{01} = 2.12$	$d_{02} = 1.87$	$d_{03} = 2.12$	$d_{04} = 2.65$
$d_{10} = 2.12$	$d_{11} = 1.41$	$d_{12} = 1.00$	$d_{13} = 1.41$	$d_{14} = 2.12$
$d_{20} = 1.87$	$d_{21} = 1.00$	$d_{22} = 0.00$	$d_{23} = 1.00$	$d_{24} = 1.87$
$d_{30} = 2.12$	$d_{31} = 1.41$	$d_{32} = 1.00$	$d_{33} = 1.41$	$d_{34} = 2.12$
$d_{40} = 2.65$	$d_{41} = 2.12$	$d_{42} = 1.87$	$d_{43} = 2.12$	$d_{44} = 2.65$

Mathematical Representation

- Each neuron i has a weight vector $\mathbf{w}_i \in \mathbb{R}^n$
- Input vector $\mathbf{x} \in \mathbb{R}^n$
- Best Matching Unit (BMU):

$$\text{BMU} = \arg\min_i \|\mathbf{x} - \mathbf{w}_i\|$$

- Weight update rule:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t) \cdot h_{ci}(t) \cdot (\mathbf{x}(t) - \mathbf{w}_i(t))$$

Competitive Learning vs. Traditional Error-Based Learning

Competitive Learning:

- **Unsupervised Learning:** Competitive learning does not require labeled data. The model organizes itself based on the data.
- **Winner-Takes-All Mechanism:** The neuron most similar to the input data (Best Matching Unit or BMU) wins and adjusts its weights. Neighboring neurons may also adjust based on their distance to the BMU.
- **Self-Organization:** The network organizes itself into clusters or maps based on input features.
- **Example:** Self-Organizing Maps (SOM)

Advantages:

- Does not require labeled data.
- Useful for clustering and dimensionality reduction.
- Helps in feature extraction and self-organizing networks.

Disadvantages:

- Limited to unsupervised tasks.
- May not perform well in direct prediction tasks.
- Sensitive to initialization and learning parameters.

Traditional Error-Based Learning

Error-Based Learning (Supervised Learning):

- **Supervised Learning:** Requires labeled data. The model is trained on input-output pairs to learn the mapping.
- **Error Minimization:** The model learns by minimizing the error between predicted and actual output (using methods like gradient descent).
- **Gradients for Update:** Weights are updated based on the gradient of the error with respect to each weight.
- **Example:** Backpropagation, linear regression, and other supervised neural networks.

Advantages:

- Effective for classification, regression, and other supervised tasks.
- Direct performance evaluation with labeled data.
- Can handle complex tasks like image recognition and natural language processing.

Disadvantages:

- Requires large labeled datasets, which can be costly to obtain.
- Prone to overfitting if not carefully managed (e.g., with regularization).
- Can be slow to converge, especially for large datasets and models.

Comparison: Competitive Learning vs. Error-Based Learning

Aspect	**Competitive Learning**	**Error-Based Learning**
Supervision	Unsupervised (No labeled data)	Supervised (Requires labeled data)
Learning Process	Winner-takes-all	Error minimization using gradient descent
Output	Data organization (Clustering)	Direct predictions or classifications
Update Mechanism	BMU (Best Matching Unit) and neighbors adjust weights	Weights adjust based on error signal
Task Examples	Clustering, Feature Mapping	Classification, Regression, Function Approximation
Advantages	No labeled data required, Self-organizing	Effective for supervised tasks, Direct performance metrics
Disadvantages	Limited to clustering, Sensitive to initialization	Requires labeled data, Can overfit

SOM Algorithm (Part 1: Initialization and BMU)

Algorithm 1 Self-Organizing Map (SOM) - Part 1

Input: Training data $X = \{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_n\}$
Parameters: Learning rate $\alpha(t)$, neighborhood radius $\sigma(t)$, map structure
Initialize weight vectors $\mathbf{w}_i$ for each neuron i
for each epoch $t = 1$ to maxEpochs **do**
    for each input vector $\mathbf{x} \in X$ **do**
        Find Best Matching Unit (BMU) c:

$$c = \arg\min_i \|\mathbf{x} - \mathbf{w}_i\|$$

▷ Use Euclidean distance

(Continue to next slide...)

    end for
end for

SOM Algorithm (Part 2: Update and Neighborhood)

Algorithm 2 Self-Organizing Map (SOM) - Part 2

Continue from previous slide...
for each neuron i **do**
    Compute neighborhood function:

$$h_{ci}(t) = \exp\left(-\frac{\|r_c - r_i\|^2}{2\sigma(t)^2}\right)$$

    Update the weight vector:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t) \cdot h_{ci}(t) \cdot (\mathbf{x} - \mathbf{w}_i(t))$$

end for

Decrease learning rate $\alpha(t)$ and neighborhood radius $\sigma(t)$

Output: Final weight vectors $\mathbf{w}_i$

Neighborhood Function $h_{ci}(t)$

- Defines how the neighbors of the BMU are updated.
- Gaussian neighborhood function:

$$h_{ci}(t) = \exp\left(-\frac{\|r_c - r_i\|^2}{2\sigma(t)^2}\right)$$

- r_c: location of BMU, r_i: location of neighbor
- $\sigma(t)$: neighborhood radius (decreases with time)

Neighborhood Function $h_{ci}(t)$

The **neighborhood function** $h_{ci}(t)$ defines how much a neuron i is influenced by the **Best Matching Unit (BMU)**. It depends on the Euclidean distance between the BMU and neuron i, and it decreases over time.

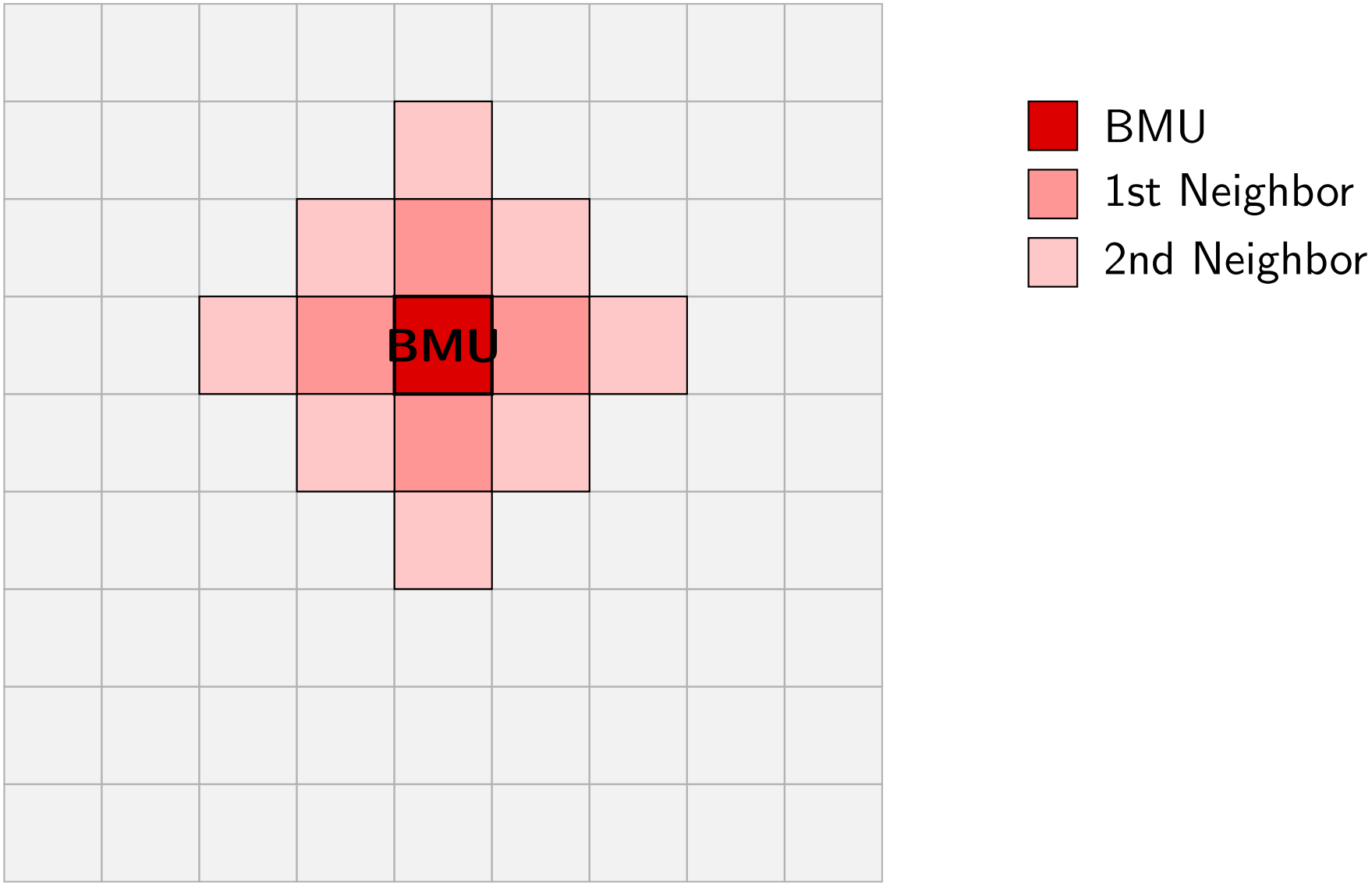
Neighborhood Function $h_{ci}(t)$

The **neighborhood function** $h_{ci}(t)$ defines how much a neuron i is influenced by the **Best Matching Unit (BMU)**. It depends on the Euclidean distance between the BMU and neuron i, and it decreases over time.
The Gaussian neighborhood function is given by:

$$h_{ci}(t) = \exp\left(-\frac{\|\mathbf{r}_c - \mathbf{r}_i\|^2}{2\sigma^2(t)}\right)$$

Where: - $\mathbf{r}_c$ and $\mathbf{r}_i$ are the coordinates of the BMU and the neighboring neuron, respectively. - $\sigma(t)$ is the radius that decays over time. - $\|\mathbf{r}_c - \mathbf{r}_i\|$ is the Euclidean distance between the BMU and neuron i.

Visualization of Neighborhood Effect (SOM)



Finding the Best Matching Unit (BMU)

Given: - Input vector $\mathbf{x} = \begin{bmatrix} 0.5 & 0.3 \end{bmatrix}$ - Weight vectors:

$$\mathbf{w}_1 = \begin{bmatrix} 0.2 & 0.4 \end{bmatrix}, \quad \mathbf{w}_2 = \begin{bmatrix} 0.6 & 0.5 \end{bmatrix}, \quad \mathbf{w}_3 = \begin{bmatrix} 0.4 & 0.7 \end{bmatrix}$$

Finding the Best Matching Unit (BMU)

Given: - Input vector $\mathbf{x} = \begin{bmatrix} 0.5 & 0.3 \end{bmatrix}$ - Weight vectors:

$$\mathbf{w}_1 = \begin{bmatrix} 0.2 & 0.4 \end{bmatrix}, \quad \mathbf{w}_2 = \begin{bmatrix} 0.6 & 0.5 \end{bmatrix}, \quad \mathbf{w}_3 = \begin{bmatrix} 0.4 & 0.7 \end{bmatrix}$$

We compute the **Euclidean distance** for each weight vector and the input:

$$d(\mathbf{x}, \mathbf{w}_1) = 0.316, \quad d(\mathbf{x}, \mathbf{w}_2) = 0.224, \quad d(\mathbf{x}, \mathbf{w}_3) = 0.404$$

Finding the Best Matching Unit (BMU)

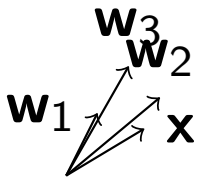
Given: - Input vector $\mathbf{x} = \begin{bmatrix} 0.5 & 0.3 \end{bmatrix}$ - Weight vectors:

$$\mathbf{w}_1 = \begin{bmatrix} 0.2 & 0.4 \end{bmatrix}, \quad \mathbf{w}_2 = \begin{bmatrix} 0.6 & 0.5 \end{bmatrix}, \quad \mathbf{w}_3 = \begin{bmatrix} 0.4 & 0.7 \end{bmatrix}$$

We compute the **Euclidean distance** for each weight vector and the input:

$$d(\mathbf{x}, \mathbf{w}_1) = 0.316, \quad d(\mathbf{x}, \mathbf{w}_2) = 0.224, \quad d(\mathbf{x}, \mathbf{w}_3) = 0.404$$

Thus, **BMU** is neuron 2.



Updating Weights of the BMU

For the **BMU** $\mathbf{w}_2$, we update its weight as follows:

$$\mathbf{w}_2(t+1) = \mathbf{w}_2(t) + \alpha(t) \cdot h_{21}(t) \cdot (\mathbf{x} - \mathbf{w}_2(t))$$

Where: - $\alpha(t) = 0.5$ - $h_{21}(t) = 0.6065$

Updating Weights of the BMU

For the **BMU** $\mathbf{w}_2$, we update its weight as follows:

$$\mathbf{w}_2(t+1) = \mathbf{w}_2(t) + \alpha(t) \cdot h_{21}(t) \cdot (\mathbf{x} - \mathbf{w}_2(t))$$

Where: - $\alpha(t) = 0.5$ - $h_{21}(t) = 0.6065$
We update the **BMU** weight:

$$\mathbf{w}_2(t+1) = \begin{bmatrix} 0.6 & 0.5 \end{bmatrix} + 0.5 \cdot 0.6065 \cdot \begin{bmatrix} -0.1 & -0.2 \end{bmatrix}$$

Updating Weights of the BMU

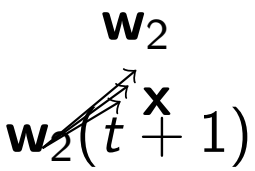
For the **BMU** $\mathbf{w}_2$, we update its weight as follows:

$$\mathbf{w}_2(t+1) = \mathbf{w}_2(t) + \alpha(t) \cdot h_{21}(t) \cdot (\mathbf{x} - \mathbf{w}_2(t))$$

Where: - $\alpha(t) = 0.5$ - $h_{21}(t) = 0.6065$
We update the **BMU** weight:

$$\mathbf{w}_2(t+1) = \begin{bmatrix} 0.6 & 0.5 \end{bmatrix} + 0.5 \cdot 0.6065 \cdot \begin{bmatrix} -0.1 & -0.2 \end{bmatrix}$$

$$\mathbf{w}_2(t+1) = \begin{bmatrix} 0.55 & 0.4 \end{bmatrix}$$



Updating Weights of Neighbors

For neighboring neurons, the weight update is given by:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t) \cdot h_{ci}(t) \cdot (\mathbf{x} - \mathbf{w}_i(t))$$

For neighbor $\mathbf{w}_1$, we compute the neighborhood effect $h_{21}(t)$.

Updating Weights of Neighbors

For neighboring neurons, the weight update is given by:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t) \cdot h_{ci}(t) \cdot (\mathbf{x} - \mathbf{w}_i(t))$$

For neighbor $\mathbf{w}_1$, we compute the neighborhood effect $h_{21}(t)$.

$$\mathbf{w}_1(t+1) = \mathbf{w}_1(t) + 0.5 \cdot 0.6065 \cdot (\mathbf{x} - \mathbf{w}_1(t))$$

$$\mathbf{w}_1(t+1) = \begin{bmatrix} 0.2 & 0.4 \end{bmatrix} + 0.5 \cdot 0.6065 \cdot \begin{bmatrix} 0.3 & -0.1 \end{bmatrix}$$

$$\mathbf{w}_1(t+1) = \begin{bmatrix} 0.32 & 0.37 \end{bmatrix}$$

Updating Weights of Neighbors

For neighboring neurons, the weight update is given by:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t) \cdot h_{ci}(t) \cdot (\mathbf{x} - \mathbf{w}_i(t))$$

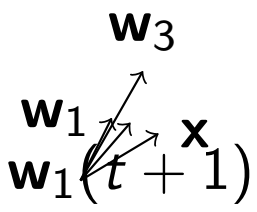
For neighbor $\mathbf{w}_1$, we compute the neighborhood effect $h_{21}(t)$.

$$\mathbf{w}_1(t+1) = \mathbf{w}_1(t) + 0.5 \cdot 0.6065 \cdot (\mathbf{x} - \mathbf{w}_1(t))$$

$$\mathbf{w}_1(t+1) = \begin{bmatrix} 0.2 & 0.4 \end{bmatrix} + 0.5 \cdot 0.6065 \cdot \begin{bmatrix} 0.3 & -0.1 \end{bmatrix}$$

$$\mathbf{w}_1(t+1) = \begin{bmatrix} 0.32 & 0.37 \end{bmatrix}$$

For **neighbor** $\mathbf{w}_3$, the update equation is the same.



Decay of Learning Rate and Radius

The learning rate and radius decay according to:

$$\alpha(t) = \alpha_0 \cdot \exp\left(-\frac{t}{\tau_1}\right)$$

$$\sigma(t) = \sigma_0 \cdot \exp\left(-\frac{t}{\tau_2}\right)$$

With initial values: - $\alpha_0 = 0.5$ - $\sigma_0 = 1.0$ - Time constants: $\tau_1 = 100$, $\tau_2 = 100$
For $t = 10$:

$$\alpha(10) = 0.5 \cdot \exp\left(-\frac{10}{100}\right) = 0.4524$$

$$\sigma(10) = 1.0 \cdot \exp\left(-\frac{10}{100}\right) = 0.9048$$

Summary of Updates

1. **BMU Selection**:

$$c = \arg\min_i \|\mathbf{x} - \mathbf{w}_i\|$$

2. **Weight Update for BMU**:

$$\mathbf{w}_i(t+1) = \mathbf{w}_i(t) + \alpha(t) \cdot h_{ci}(t) \cdot (\mathbf{x} - \mathbf{w}_i(t))$$

3. **Learning Rate and Radius Decay**:

$$\alpha(t) = \alpha_0 \cdot \exp\left(-\frac{t}{\tau_1}\right), \quad \sigma(t) = \sigma_0 \cdot \exp\left(-\frac{t}{\tau_2}\right)$$

4. **Neighborhood Update**: The weights of neighbors are updated with the same formula as the BMU, but the influence decreases with distance from the BMU.

Pitfalls of SOM

- Convergence can be slow.
- Sensitive to initialization.
- Requires tuning of hyperparameters (learning rate, neighborhood size).
- Fixed grid size limits scalability.
- Difficult to interpret in high dimensions.

Conclusion

- SOM is a powerful unsupervised technique for clustering and visualization.
- Preserves topology and structure of high-dimensional data.
- Requires careful tuning and understanding of limitations.